

Übungsblatt 2: Floating-Point Arithmetic (Gleitkommaarithmetik)

zum Vorlesungsskript „Numerical Methods“ (Prof. Mark Schutera), Kapitel 2

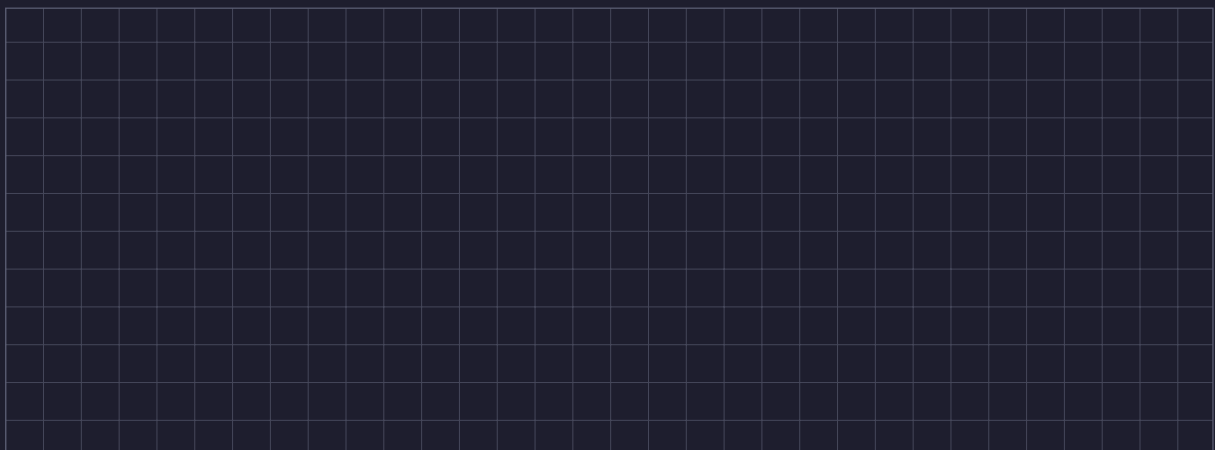
13. Juni 2026

Dieses Übungsblatt deckt Kapitel 2 des Skripts ab: Fehlerarten, Gleitkommadarstellung (floating-point representation) nach IEEE 754, Maschinengenauigkeit (machine epsilon), Festkommadarstellung (fixed-point representation) sowie Auslöschung (loss of significance / catastrophic cancellation). **Gesamtpunktzahl: 38 Punkte.** Die Aufgaben steigen im Schwierigkeitsgrad an: Aufgaben 1–3 prüfen das Begriffsverständnis, Aufgaben 4–7 sind Standardaufgaben analog zum Skript, Aufgabe 8 ist eine Transferaufgabe. Die Lösungen befinden sich ab Seite 7 — erst nach eigener Bearbeitung lesen!

Aufgaben

Aufgabe 1 (4 Punkte): Fehlerarten erkennen und zuordnen

- (a) Definieren Sie kurz die drei im Skript unterschiedenen Fehlerarten: Modellierungsfehler (modeling error), Abschneide-/Trunkierungsfehler (truncation error) und Rundungsfehler (rounding error). (2 Punkte)
- (b) Ordnen Sie die folgenden Szenarien jeweils genau einer der drei Fehlerarten zu und begründen Sie kurz: (2 Punkte)
 - (i) Ein schwingendes Pendel wird in einer Simulation als reibungsfrei angenommen, obwohl in der Realität Luftwiderstand wirkt.
 - (ii) Die unendliche Reihe $e^x = \sum_{k=0}^{\infty} \frac{x^k}{k!}$ wird zur Berechnung nach dem zehnten Summanden abgebrochen.
 - (iii) Die Zahl $\frac{1}{3}$ wird im Computer als 0.33333333 gespeichert.



Aufgabe 2 (4 Punkte): Warum $10 \div 3$ nicht exakt darstellbar ist

- Erklären Sie anhand der schriftlichen Division, warum $\frac{10}{3}$ mit endlich vielen Dezimalziffern niemals exakt aufgeschrieben oder gespeichert werden kann. (1 Punkt)
- Berechnen Sie den Rundungsfehler (rounding error), der entsteht, wenn man $\frac{10}{3}$ nach drei Nachkommastellen abschneidet (truncate), also als 3.333 speichert. Geben Sie den Fehler auch als Bruch an. (2 Punkte)
- Geben Sie an, wie der Wert $\frac{10}{3}$ in den folgenden vier Darstellungstypen gespeichert würde: Integer (int), Floating-point (float, 7 Nachkommastellen), Double precision (double, 16 Nachkommastellen), Fixed-point mit 4 Nachkommastellen. (1 Punkt)

Aufgabe 3 (3 Punkte): Relative vs. absolute Präzision

- (a) Erklären Sie den Unterschied zwischen der relativen und der absoluten Präzision von Gleitkommazahlen. Welche der beiden ist (näherungsweise) konstant, welche hängt von der Größe der dargestellten Zahl ab? Geben Sie die zugehörigen Ausdrücke aus dem Skript an. (1.5 Punkte)
- (b) Das Skript veranschaulicht den Unterschied mit dem Vergleich $1 : 2 = 0.5$, aber $10 : 2 = 5$. Erläutern Sie, was diese Randnotiz über Schritte (relative Präzision) und Lücken (gaps, absolute Präzision) aussagt. (0.5 Punkte)
- (c) Berechnen Sie näherungsweise die absolute Präzision (Lückengröße zwischen benachbarten darstellbaren Zahlen) in IEEE 754 single precision ($\varepsilon_{\text{mach}} \approx 1.19 \times 10^{-7}$) für $x = 1$ und für $x = 10^6$. (1 Punkt)

Aufgabe 4 (6 Punkte): IEEE 754 single precision: Dekodieren und Kodieren

In IEEE 754 single precision (32 Bit: 1 Vorzeichenbit + 8 Exponentenbits + 23 Mantissenbits) wird eine normalisierte Zahl gespeichert als

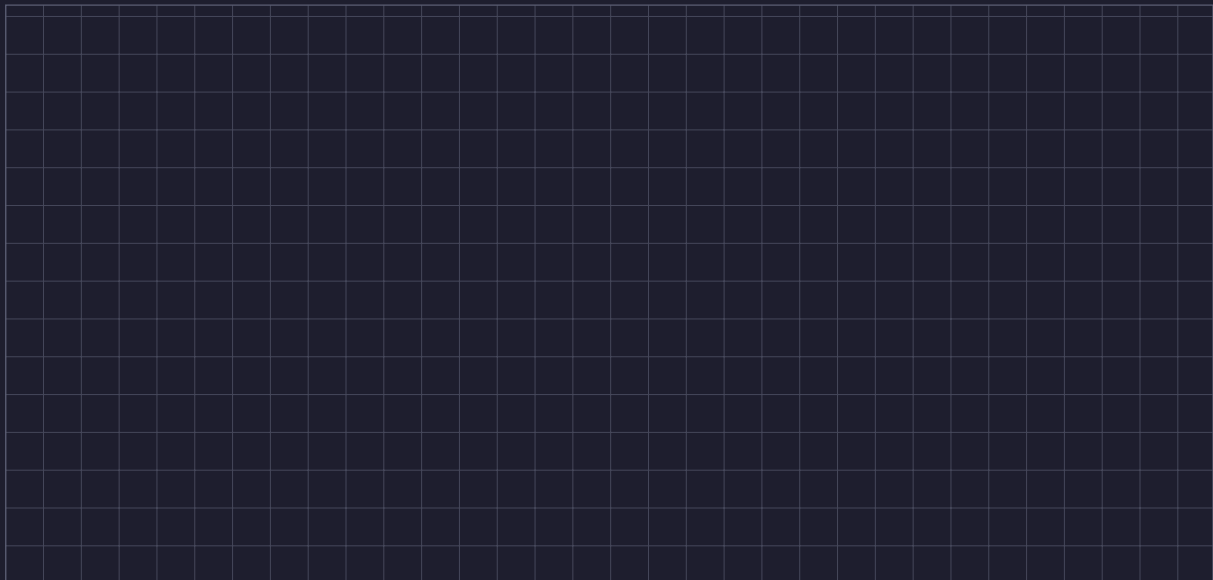
$$x = (-1)^s \cdot 1.m \cdot 2^{E-127},$$

wobei s das Vorzeichenbit (sign bit), m das Mantissenfeld (mantissa) und E der gespeicherte Exponent mit Bias 127 ist (die führende 1 der normalisierten Mantisse wird nicht mitgespeichert). *Hinweis: Die Konvention des impliziten führenden Mantissenbits geht leicht über die Skriptdarstellung (S. 19–20) hinaus und ist hier vollständig angegeben — eine kleine Transferkomponente.*

- (a) **Dekodieren:** Welche Dezimalzahl ist hier gespeichert? (3 Punkte)

$$s = 1, \quad E = 10000010_2, \quad m = 10100000000000000000000_2$$

- (b) **Kodieren:** Bestimmen Sie Vorzeichenbit s , Exponentenfeld E (binär, 8 Bit) und Mantissenfeld m (binär, 23 Bit) für die Dezimalzahl $x = 6.25$. (3 Punkte)



Aufgabe 5 (5 Punkte): Machine Epsilon und die Mantisse

- (a) Definieren Sie Machine Epsilon ($\varepsilon_{\text{mach}}$, Maschinengenauigkeit) und geben Sie die Formel in Abhängigkeit von der Anzahl t der Mantissenbits an. Berechnen Sie daraus $\varepsilon_{\text{mach}}$ für IEEE 754 single precision ($t = 23$) und double precision ($t = 52$). (2 Punkte)
- (b) Betrachten Sie eine (unnormalisierte) 2-Bit-Mantisse mit Significand $0.xx_2$. Listen Sie alle vier Bitkombinationen mit ihren Dezimalwerten auf und geben Sie das zugehörige Machine Epsilon an. (2 Punkte)
- (c) Wie viele verschiedene Werte zwischen zwei benachbarten Zweierpotenzen erlaubt eine Mantisse mit t Bits allgemein? Wie viele sind es konkret bei $t = 3$? (1 Punkt)

Aufgabe 6 (4 Punkte): Constructing scale: Zehnerpotenzen in Binärdarstellung

Stellen Sie die Zehnerpotenzen 10^1 , 10^2 und 10^3 jeweils

- als Binärzahl dar, (1.5 Punkte)
- in normalisierter Form $1.xxx_2 \times 2^e$ dar, (1.5 Punkte)
- und geben Sie das zugehörige gespeicherte Exponentenfeld E (binär, 8 Bit, Bias 127, IEEE 754 single precision) an. (1 Punkt)

- (d) Worin unterscheidet sich die Präzision der Festkommadarstellung grundsätzlich von der der Gleitkommadarstellung? (1 Punkt)

Aufgabe 8 (8 Punkte): Transfer: Auslöschung, Overflow/Underflow und Quantisierung

- (a) Eine Maschine speichert Zahlen mit 8 signifikanten Stellen (kaufmännisch gerundet). Gegeben sind die beiden Paare

(i) $a = 12345678.5$, $b = 12345678.0$ und (ii) $a = 12345678.4$, $b = 12345678.0$.

Berechnen Sie für beide Paare: die gespeicherten Werte $\tilde{a}$, $\tilde{b}$, das Maschinenergebnis $\tilde{a} - \tilde{b}$, die wahre Differenz $a - b$ sowie den absoluten und den relativen Fehler des Maschinenergebnisses. Was fällt beim Vergleich der beiden Paare auf? (3 Punkte)

- (b) Betrachten Sie die Funktion $f(\epsilon) = \frac{1}{1 + \epsilon - 1}$. Was liefert eine Auswertung in IEEE 754 double precision ($\varepsilon_{\text{mach}} \approx 2.22 \times 10^{-16}$) für $\epsilon = 10^{-20}$, wenn der Nenner exakt so wie angegeben berechnet wird — und warum? Geben Sie eine mathematisch äquivalente Umformulierung an, die das Problem vollständig vermeidet, und werten Sie diese für $\epsilon = 10^{-20}$ aus. (3 Punkte)

- (c) Die größte darstellbare Zahl in IEEE 754 single precision liegt bei ca. 10^{38} , gesetzt durch den größten Exponenten $e = +127$. Was geschieht in single precision bei der Berechnung von x^2 für $x = 10^{20}$ und bei der Berechnung von y^2 für $y = 10^{-25}$? Benennen Sie beide Phänomene und beschreiben Sie eine gefährliche Folgerechnung, die sich aus dem zweiten Fall ergeben kann. (1.5 Punkte)

- (d) Quantisierte Modelle (quantized models) auf Edge-Geräten rechnen mit niedrigpräziser Arithmetik (z. B. 8-Bit-Integer); bei binären neuronalen Netzen (binary neural networks, BNNs) sind Gewichte und Aktivierungen sogar auf $\{-1, +1\}$ beschränkt. Erläutern Sie kurz, warum gerade dort das Verständnis von Gleitkommadarstellung und Rundungseffekten kritisch ist. (0.5 Punkte)

Lösungen — erst nach Bearbeitung lesen!

Lösung zu Aufgabe 1

- (a)
- **Modellierungsfehler (modeling error):** entsteht, weil alle Modelle Vereinfachungen der Realität sind; die Differenz zwischen Modell und realem System führt einen Fehler ein — der feine Unterschied zwischen dem, was wir $f(x)$ *nennen*, und dem, was tatsächlich ein $f(x)$ *ist*.
 - **Abschneide-/Trunkierungsfehler (truncation error):** entsteht, wenn ein unendlicher Prozess (Reihe, Grenzwert, kontinuierliche Funktion) durch einen endlichen approximiert wird — auch Approximationsfehler (approximation error) genannt.
 - **Rundungsfehler (rounding error):** Differenz zwischen dem wahren Wert und dem Wert, den man nach dem Abschneiden der (Dezimal-)Entwicklung erhält; entsteht, weil Computer Zahlen stets mit endlich vielen Ziffern bzw. Bits speichern.
- (b)
- (i) **Modellierungsfehler:** Das Modell (reibungsfreies Pendel) vereinfacht die Realität (mit Luftwiderstand); der Fehler entsteht bereits vor jeder Berechnung in der Modellbildung.
 - (ii) **Trunkierungsfehler:** Ein unendlicher Prozess (unendliche Reihe) wird durch einen endlichen (10 Summanden) ersetzt.
 - (iii) **Rundungsfehler:** Der Speicherwert 0.33333333 schneidet die unendliche Dezimalentwicklung $0.\overline{3}$ nach endlich vielen Ziffern ab; der Fehler entsteht durch die endliche Speicherdarstellung.

Lösung zu Aufgabe 2

- (a) Die schriftliche Division liefert in jedem Schritt dasselbe Muster: $10 \div 3 = 3$, Rest 1; nach „bring down a 0“ wieder $10 \div 3 = 3$, Rest 1 — und so weiter. Der Rest wird nie null, also wiederholt sich die Ziffer 3 unendlich oft: $\frac{10}{3} = 3.333 \dots = 3.\overline{3}$. Da man beim Aufschreiben oder Speichern irgendwo stoppen muss, entsteht zwangsläufig ein Fehler — egal, wie viele Ziffern man schreibt.

(b)

$$\text{Rundungsfehler} = |3.333333 \dots - 3.333| = 0.000333 \dots$$

Als Bruch:

$$\frac{10}{3} - \frac{3333}{1000} = \frac{10000 - 9999}{3000} = \frac{1}{3000} = 0.000\overline{3}.$$

	Darstellung	gespeicherter Wert
	Integer (int)	3
(c)	Floating-point (float)	3.3333333
	Double precision (double)	3.3333333333333333
	Fixed-point (4 Nachkommastellen)	3.3333

Lösung zu Aufgabe 3

- (a) Die **relative Präzision** von Gleitkommazahlen ist näherungsweise konstant: Sie beträgt ca. 2^{-t} (mit t = Anzahl der Mantissenbits), unabhängig von der Größe der dargestellten Zahl. Die **absolute Präzision** hängt dagegen von der Größenordnung der Zahl ab:

$$\text{absolute Präzision} = \varepsilon_{\text{mach}} \cdot |x|.$$

Der absolute Abstand benachbarter darstellbarer Zahlen wächst also proportional zu $|x|$: Große Zahlen haben größere absolute Lücken (gaps) als kleine Zahlen.

- (b) Wird das Intervall $[0, 1]$ in zwei Schritte geteilt, beträgt die Schrittweite 0.5; wird das Intervall $[0, 10]$ ebenfalls in zwei Schritte geteilt, beträgt sie 5. Gleiche *Anzahl* von Schritten (gleiche relative Präzision), aber Lücken von 0.5 vs. 5 — die absolute Präzision skaliert mit der Größe der Zahl.
- (c) Mit $\varepsilon_{\text{mach}} \approx 1.19 \times 10^{-7}$:

$$\begin{aligned} x = 1 : \quad \varepsilon_{\text{mach}} \cdot |x| &\approx 1.19 \times 10^{-7} \cdot 1 = 1.19 \times 10^{-7}, \\ x = 10^6 : \quad \varepsilon_{\text{mach}} \cdot |x| &\approx 1.19 \times 10^{-7} \cdot 10^6 = 0.119 \approx 0.12. \end{aligned}$$

Bei $x = 10^6$ liegen benachbarte darstellbare Zahlen also bereits gut eine Zehntel-Einheit auseinander, während die Lücke bei $x = 1$ nur etwa 10^{-7} beträgt.

Lösung zu Aufgabe 4

- (a) **Dekodieren.** Gespeicherter Exponent:

$$E = 10000010_2 = 128 + 2 = 130_{10}, \quad e = E - 127 = 130 - 127 = 3.$$

Significand (mit impliziter führender 1): das Mantissenfeld beginnt mit 101, alle weiteren Bits sind null, also

$$1.101_2 = 1 + 1 \cdot 2^{-1} + 0 \cdot 2^{-2} + 1 \cdot 2^{-3} = 1 + 0.5 + 0.125 = 1.625.$$

Mit Vorzeichenbit $s = 1$:

$$x = (-1)^1 \cdot 1.625 \cdot 2^3 = -1.625 \cdot 8 = -13.0.$$

Probe: $13_{10} = 1101_2 = 1.101_2 \times 2^3$. ✓

- (b) **Kodieren von $x = 6.25$.** Umwandlung ins Binärsystem:

$$6.25 = 4 + 2 + 0.25 = 110.01_2.$$

Normalisieren (Komma um zwei Stellen nach links verschieben):

$$110.01_2 = 1.1001_2 \times 2^2 \quad \Rightarrow \quad e = 2.$$

Damit:

$s = 0$ (Zahl ist positiv),

$$E = e + 127 = 2 + 127 = 129_{10} = 10000001_2,$$

$m = 1001000000000000000000_2$ (Nachkommabits 1001, mit Nullen auf 23 Bit aufgefüllt).

Probe: $(-1)^0 \cdot 1.1001_2 \cdot 2^2 = (1 + 0.5 + 0.0625) \cdot 4 = 1.5625 \cdot 4 = 6.25$. ✓

Lösung zu Aufgabe 5

- (a) Machine Epsilon ist die kleinste positive Zahl, sodass $1 + \varepsilon_{\text{mach}} \neq 1$ in der Computerarithmetik gilt; es quantifiziert die obere Schranke des relativen Fehlers durch Rundung in der Gleitkomma-Arithmetik. Formel:

$$\varepsilon_{\text{mach}} = 2^{-t},$$

mit t = Anzahl der Mantissenbits. Konkret:

$$\begin{aligned} \text{single precision } (t = 23) : \quad \varepsilon_{\text{mach}} &= 2^{-23} = \frac{1}{8388608} \approx 1.19 \times 10^{-7}, \\ \text{double precision } (t = 52) : \quad \varepsilon_{\text{mach}} &= 2^{-52} \approx 2.22 \times 10^{-16}. \end{aligned}$$

- (b) Alle vier Bitkombinationen der unnormalisierten 2-Bit-Mantisse $0.xx_2$:

$$00 : 0.00_2 = 0 + 0 \times 2^{-1} + 0 \times 2^{-2} = 0.0,$$

$$01 : 0.01_2 = 0 + 0 \times 2^{-1} + 1 \times 2^{-2} = 0.25,$$

$$10 : 0.10_2 = 0 + 1 \times 2^{-1} + 0 \times 2^{-2} = 0.5,$$

$$11 : 0.11_2 = 0 + 1 \times 2^{-1} + 1 \times 2^{-2} = 0.75.$$

Machine Epsilon: $\varepsilon_{\text{mach}} = 2^{-2} = 0.25$ — genau der Abstand zwischen zwei benachbarten Mantissenwerten.

- (c) Eine Mantisse mit t Bits erlaubt 2^t verschiedene Werte zwischen zwei benachbarten Zweierpotenzen (skaliert durch den Exponenten). Für $t = 3$: $2^3 = 8$ Werte.

Lösung zu Aufgabe 6

- (a) Binärdarstellungen:

$$10^1 = 10_{10} = 1010_2 \quad (8 + 2 = 10),$$

$$10^2 = 100_{10} = 1100100_2 \quad (64 + 32 + 4 = 100),$$

$$10^3 = 1000_{10} = 1111101000_2 \quad (512 + 256 + 128 + 64 + 32 + 8 = 1000).$$

- (b) Normalisierte Form (Komma hinter die führende 1 schieben):

$$10^1 = 1.010_2 \times 2^3,$$

$$10^2 = 1.100100_2 \times 2^6,$$

$$10^3 = 1.111101000_2 \times 2^9.$$

- (c) Gespeicherter Exponent $E = e + 127$:

$$e = 3 : E = 130_{10} = 10000010_2 \quad (128 + 2 = 130),$$

$$e = 6 : E = 133_{10} = 10000101_2 \quad (128 + 4 + 1 = 133),$$

$$e = 9 : E = 136_{10} = 10001000_2 \quad (128 + 8 = 136).$$

Lösung zu Aufgabe 7

- (a) Gespeichert wird $1.234567 \cdot 10^6 = 1234567$.

- (b) Die kleinste darstellbare Differenz ist ein Integer-Schritt, zurückskaliert mit 10^{-6} :

$$\text{Präzision} = 10^{-6} = 0.000001.$$

Der maximale Rundungsfehler beim Speichern ist ein halber Schritt:

$$0.5 \times 10^{-6} = 0.0000005.$$

- (c) Größter zu speichernder Betrag: $1000 \cdot 10^6 = 10^9 = 1\,000\,000\,000 < 2\,147\,483\,647$ — passt (ebenso $-10^9 > -2\,147\,483\,648$). Mit Skalierungsfaktor 10^7 wäre dagegen $1000 \cdot 10^7 = 10^{10} = 10\,000\,000\,000 > 2\,147\,483\,647$ — Überlauf des Integer-Bereichs. 10^6 ist also die größte Zehnerpotenz, die als Skalierungsfaktor möglich ist.

- (d) Bei der Festkommadarstellung ist die Lücke zwischen darstellbaren Zahlen (die absolute Präzision) *immer gleich*, egal wie groß oder klein der Wert ist — das gibt mehr Kontrolle und vorhersagbare Präzision und Performance. Bei der Gleitkommadarstellung ist dagegen die *relative* Präzision näherungsweise konstant ($\approx 2^{-t}$), während die absolute Lücke proportional zu $|x|$ wächst — dafür ist der darstellbare Wertebereich (Dynamikumfang) viel größer.

Lösung zu Aufgabe 8

- (a) **Paar (i):** $a = 12345678.5$, $b = 12345678.0$. Mit 8 signifikanten Stellen (kaufmännisch gerundet, .5 rundet auf):

$$\tilde{a} = 12345679, \quad \tilde{b} = 12345678.$$

Maschinenergebnis: $\tilde{a} - \tilde{b} = 1$. Wahre Differenz: $a - b = 0.5$.

$$\text{absoluter Fehler} = |1 - 0.5| = 0.5, \quad \text{relativer Fehler} = \frac{0.5}{0.5} = 1 = 100\%.$$

Paar (ii): $a = 12345678.4$, $b = 12345678.0$. Mit 8 signifikanten Stellen (12345678.4 rundet ab):

$$\tilde{a} = 12345678, \quad \tilde{b} = 12345678.$$

Maschinenergebnis: $\tilde{a} - \tilde{b} = 0$. Wahre Differenz: $a - b = 0.4$.

$$\text{absoluter Fehler} = |0 - 0.4| = 0.4, \quad \text{relativer Fehler} = \frac{0.4}{0.4} = 1 = 100\%.$$

Beobachtung: Die wahren Resultate unterscheiden sich nur um 0.1 (0.5 vs. 0.4), die Maschinenresultate aber um eine ganze Einheit (1 vs. 0) — in beiden Fällen beträgt der relative Fehler 100 %. Beim Subtrahieren nahezu gleicher Zahlen heben sich die führenden (korrekten) Ziffern auf, und übrig bleiben nur die rundungsfehleranfälligen letzten Stellen: Auslöschung (loss of significance / catastrophic cancellation) verstärkt den Rundungsfehler massiv.

- (b) Da $\epsilon = 10^{-20} < \epsilon_{\text{mach}} \approx 2.22 \times 10^{-16}$, wird die Addition $1 + \epsilon$ in double precision exakt auf 1 gerundet ($1 + \epsilon$ und 1 sind nicht unterscheidbar). Der Nenner wird damit

$$\underbrace{(1 + \epsilon)}_{\rightarrow 1} - 1 = 0,$$

und die Auswertung liefert eine Division durch null (in IEEE 754: $+\infty$) statt des wahren Werts. **Umformulierung:** Der Nenner vereinfacht sich analytisch zu $1 + \epsilon - 1 = \epsilon$, also

$$f(\epsilon) = \frac{1}{\epsilon}.$$

Diese Form enthält keine Subtraktion nahezu gleicher Zahlen mehr und liefert für $\epsilon = 10^{-20}$ korrekt

$$f(10^{-20}) = \frac{1}{10^{-20}} = 10^{20}.$$

- (c) Für $x = 10^{20}$ ist $x^2 = 10^{40} > 10^{38}$, also größer als die größte darstellbare Zahl in single precision: **Overflow** — das Ergebnis wird als $+\infty$ approximiert. Für $y = 10^{-25}$ ist $y^2 = 10^{-50}$, eine Zahl so nah an null, dass sie in single precision auf 0 gerundet wird: **Underflow**. Gefährliche Folgerechnung: Wird anschließend z. B. y/y^2 gebildet, dividiert man durch die gespeicherte 0 statt durch 10^{-50} — das Ergebnis wird ∞ (bzw. ein Laufzeitfehler) statt des wahren Werts $10^{-25}/10^{-50} = 10^{25}$, der in single precision problemlos darstellbar wäre ($10^{25} < 10^{38}$). Der Schaden entsteht hier also allein durch den Underflow im Nenner.
- (d) Bei niedrigpräziser Arithmetik sind Machine Epsilon groß und die darstellbaren Wertemengen extrem klein (bei BNNs nur $\{-1, +1\}$). Rundungs- und Quantisierungsfehler sind dadurch um viele Größenordnungen größer als bei float/double, können sich in den vielen aufeinanderfolgenden Operationen eines neuronalen Netzes verstärken und akkumulieren und so das Inferenzergebnis spürbar verfälschen. Wer Modelle on the edge einsetzt, muss daher verstehen, welche Werte überhaupt darstellbar sind, wo gerundet wird und wie sich diese Fehler fortpflanzen — sonst lassen sich Genauigkeitsverluste durch Quantisierung weder vorhersagen noch kontrollieren.